Compte Rendu #4

Mathys CHAMPENOIS, Cesar CHARBEY, Mickael CZOLACZ - Super Pixels

Dépôt GitHub : `https://github.com/Mathys3114/ProjetCCMTI`

1 Bilan des Réalisations

Cette semaine, nous avons finalisé le pipeline complet allant de l'image brute au fichier compressé propriétaire. L'intégration de la phase de connectivité et d'un système de stockage binaire optimisé nous permet d'atteindre des taux de compression significatifs tout en préservant une qualité visuelle élevée.

(a) Image d'entrée (b) Image traitée

FIGURE 1 – Comparaison de l'image pré-traitement et post-traitement. On observe une très bonne fidélité entre les deux expliquant le PSNR élevé

Performances Clés (Image de test Parrot)

- **Taille d'origine (PPM) :** 3 275 536 octets
- **Taille compressée final (.slic) :** 325 945 octets
- **Facteur de compression :** $\approx 10\times$
- **Qualité (PSNR) :** 30.74 dB (avec $K = 2048$, $m = 2$)

2 Architecture du Format de Compression

Le gain en espace de stockage (passage de 1.5 Mo à 325 Ko) repose sur deux piliers techniques développés.

2.1 Le "Bit-Packing" des Indices

Plutôt que de stocker l'indice du superpixel sur un type de donnée standard (8 ou 32 bits), nous calculons le nombre de bits n strictement nécessaire : $n = \lceil \log_2(K) \rceil$. La structure `bitFile` gère ce flux au niveau du bit :

— Les indices sont concaténés dans l'accumulateur `buf`.
— Dès que 8 bits sont accumulés, l'octet est écrit dans le flux binaire.
— Cette méthode élimine tout "bit perdu" dû à l'alignement mémoire standard.

2.2 Run-Length Encoding (RLE)

En observant la forte redondance spatiale (les pixels adjacents appartiennent souvent au même superpixel), nous avons appliqué un codage RLE par ligne. Au lieu de stocker chaque indice, nous stockons des couples *(indice, répétition)*. Cette optimisation a divisé par 5 la taille du fichier final par rapport à un stockage brut des indices packés.